Decomposition Grammar

Preface

This is a working draft, not a complete text. Despite making what I consider to be considerable progress, I am, as a non-linguist, very much out of my depth. I have been encouraged to keep going by the consistency by which my model generated predictions that turned out to be true, non-trivial, and often considered unexplained by other frameworks. These include the endogenous explanation of the symptom profiles of Wernicke's, Broca's, and Conduction aphasia, as well as the tight correlation between head-final languages and topic-prominence.

There is quite a lot that I believe will be difficult to complete on my own, and for which collaboration with someone with formal linguistics training would be invaluable. While any sort of feedback is appreciated, a referral to any academic who would be interested and capable of collaborating on this project might help to turn this into something publishable. The sections that are incomplete have been clearly marked.

I will also note that my terminology usage will likely be non-standard, due to my status as an outsider. This was inevitable to some degree, but I have done my best to conform to linguistics conventions where possible, and to mark my own specific terminology where existing conventions were unavailable.

Abstract

We propose a novel model for speech production and reception termed Decomposition Grammar. This model presents an alternative mechanistic account of language production, reception, and acquisition, replacing Merge with its inverse, Split, as the core operation of fluent grammar production, and deriving the result that there are two distinct grammatical systems running in parallel in unimpaired individuals. We label these two systems the "Tree System", which is the primary system of spontaneous speech, and the "Linear System", which is the primary system of speech reception and monitoring. We argue that the grammatical sentences admitted by the Linear System form a strict superset of those generated by the Tree System.

DG is built as a precise and mechanistic model of language based on the observation that human working memory is deeply limited, and showing that two systems are required to handle the asymmetric space requirements of production and reception.

We derive a number of unusual results, such as the idea that the grammar of the Linear System is a fully Context Sensitive Grammar, and the Tree System parses from right-to-left, that are nonetheless consistent within the model. Our model provides a simple and precisely specified explanation of: the precise comprehension and production deficit profiles associated with Broca's aphasia, Wernicke's Aphasia, and Conduction aphasia; the existence of non-CFG structures in natural languages; the rarity of non-CFG structures across global languages; the association of non-CFG structures with marked speech; the discrepancy between meta-linguistic intuitions and generative accounts of grammar; the rigidity of acquisition order across global languages; the close relationship between left-branching languages and topic-prominence; the existence of pragmatics and strongly preferred word orders; the relationship between left-branching and working-memory failures; the gap between fluency in reception versus production; the parsing issues associated with centre-embedding and garden-path sentences; and the interlanguage fossilisation phenomenon, with implications for psycholinguistics and neurolinguistics.

Contents

1 Motivation

1.1 Universal Grammar and Generative Grammar

Chomsky's Transformation Grammar began the ongoing search for a comprehensive theory of Universal Grammar. However, following developments seem to have only made the nature of a purported universal grammar murkier rather than clearer. The original formulation of Transformation Grammar was shockingly predictive, in that it seemed to correctly describe almost all grammatically correct human language as belonging to sequences producible by a Context Free Grammar (CFG).

It was known from early on that humans often produced sentences that were not CFG, but that these sentences were often considered ungrammatical. This in part motivated the formal distinction between Performance and Competence — the central claim being that the language produced by speakers was unreliable, but their grammaticality judgements were consistent with their language competence.

This distinction seemed to save earlier formulations of Generative Grammar until it was confirmed that certain languages accept structures such as Right Node Raising (RNR) and Cross Serial Dependencies (CSD) as grammatical despite their formal incompatibility with CFG. Generative Grammar has responded to these challenges by proposing alternative algorithms by which Universal Grammar could be produced, but all of them require more complex mechanisms than CFG.

We believe that this line of inquiry has been unproductive, for reasons including but not limited to those outlined below:

1. **The unreliability of grammaticality judgements.** It is well observed that humans have the capacity to judge the grammaticality of sentences for 'normal' sentences — English speakers can easily determine that "He went to work" is grammatical and "He go to work" is not. However, there are a great number of forms for which speakers provide inconsistent grammaticality judgements *[citation]*.

2. **The discrepancy between reported parsing and generative mechanisms.** Regardless of generative grammar's accuracy, it is noteworthy that although humans have been formalising grammar for thousands of years, none until Chomsky proposed a generative tree system. Native speakers are more than capable of describing how they interpreted the grammar of a received sentence, and they rarely if ever describe a generative grammar. Often, the reported parsing mechanism is formally at odds with generative grammar entirely *[See note on Chinese Pivotal Sentences — to be added]*.

This discrepancy makes the mechanism of grammaticality judgements very unclear. It either presupposes that we have an intrinsic function for assessing grammaticality that is entirely different to how we actually process speech, or that the way speakers describe grammar parsing is something of a hallucination unrelated to an unconscious latent generative grammar parsing system. Both explanations are uncomfortable: the former would need to explain why we would have evolved this function for assessing grammaticality despite not having an obvious survival purpose, while the latter would need to explain why we evolved a separate system for hallucinatory grammar parsing.

3. **Theory bloat.** The generative mechanisms required to make features such as RNR and Cross Serial Dependency compatible with generative grammar require the theory to become much more complex than its original formulation. Transformation Grammar was powerful in large part due to its simplicity, but this simplicity has been degraded over time.

4. **Universal Grammar should be universal.** If we are proposing that the syntactic features of all languages share the same properties, then all languages should share those properties. That is, if all humans are endowed with the innate capacity to produce cross serial dependencies, it would be strange to imagine that only speakers of Swiss German and a handful of other languages use it. We should desire a theory that not only accounts for all features in all natural languages, but also demonstrates that the mechanisms required to produce these features are used by all natural languages.

We are not the first to raise these criticisms of Generative Grammar. However, the persistence of the Generative Grammar tradition has not been without reason: it seems very close to a perfect theory. The great bulk of observed grammatical speech does conform to the simple rules outlined in Transformation Grammar; the exceptions are rare, typically highly marked, and often considered strange by speakers *[citation]*.

This apparent paradox — that non-CFG structures are possible but strange — is one that any effective theory of grammar must account for. Numerous mechanisms have been proposed, including cognitive difficulties, but there is not yet a consensus as to how these cognitive mechanisms work or what their restrictions are *[citation; expand on alternatives to non-generative UG theories]*.

1.2 Broca's and Wernicke's Aphasia

It has long been established that Broca's area and Wernicke's area both play central roles in our grammatical faculties. However, no clear consensus has emerged on what exactly these roles are. Intuitive assumptions that one is responsible for grammar and the other for lexicon, or that one is responsible for reception and the other for production, do not line up with the observed language patterns of patients with Broca's or Wernicke's aphasia.

Specifically, Wernicke's patients have marked difficulties with both production and reception in both grammatical and lexical aspects. Meanwhile, Broca's patients have great difficulty with production while reception is largely unaffected — however, they still display consistent difficulties with specific sentence constructions such as passive forms.

This discrepancy poses a significant problem for any attempt to model the roles of these two areas in human language. We should desire that Broca's area and Wernicke's area have distinct roles; however, the idea that Broca's area is only used for grammaticality for a certain subset of sentences is strange and eludes easy explanation.

This is further complicated by observations that Broca's area continuously operates during normal speech reception *[citation]*. That is, it seems to be doing something even for comprehension tasks for which it is demonstrably unnecessary, as Broca's patients do not typically struggle with normal speech reception. A successful neurolinguistic theory must account for this. We argue that existing grammatical frameworks have yet to produce a satisfying account. Specifically, we desire that Broca's area's function be one that is activated during all reception but is not central to comprehension except in edge cases.

These are very tight constraints, and Decomposition Grammar is our attempt at defining these functions. We will show that DG provides a clean account of these discrepancies.

2 Overview of Decomposition Grammar

Decomposition Grammar (DG) is centred around demonstrating the following claim: non-impaired individuals have two distinct linguistic systems operating in parallel. We term these the *Tree System* and the *Linear System*.

Both systems are employed in both reception and production in distinct but clearly delineated ways.

The **Tree System** is centred on Broca's area. It is the system that individuals

with Broca's aphasia lack, and its absence accounts for the symptoms of Broca's aphasia in both production and reception. It is responsible for fluent production, fluent language acquisition, and has a secondary role in reception for sequences for which the Linear System is not appropriate. This system exclusively operates on sequences generatable by a Context Free Grammar. Its core operations are SPLIT and MERGE, defined in Section 3. An important property of the Tree System is that its operations are largely — if not entirely — unconscious.

The **Linear System** utilises various brain regions, but not Broca's area. It does not employ MERGE or SPLIT. It is the primary system for speech reception, speech monitoring, and monitored speech, and plays a separate but distinct role in language acquisition. The Linear System is the only system that patients with severe Broca's aphasia have access to. We will demonstrate that the Linear System interprets sentences generatable by a Context Sensitive Grammar. Unlike the Tree System, the operations of the Linear System are consciously accessible — it accounts for the operations of language reception and production that the individual has conscious access to.

We note that while the two systems are disjoint, they operate on the same core object of *semantic chunks*, which are encoded with *traces* — markers that express a chunk's grammatical role. The common features of the grammars produced by the two systems are a consequence of this feature. That is, the two grammars are the result of two different systems that have access to the same syntactic markers but apply them under radically different parsing mechanisms. In addition, both systems have access to MAP and UNMAP, which we claim is a function of — and perhaps the primary function of — Wernicke's area.

We argue that the structural differences between the Tree System and the Linear System provide a simple and mechanistic account of problematic structures such as RNR. The Linear System admits a superset of structures produced by the Tree System: it is not constrained by CFG, and is thus capable of judging as grammatical sentences that CFG could never produce. This predicts that non-CFG sentences exist, but are marked and rare since they cannot be produced except via monitored speech, as monitoring requires use of the Linear System. A core prediction of Decomposition Grammar is therefore that novel and deeply structured non-CFG structures are never produced in spontaneous speech in any language.

Unlike many theories of grammar, acquisition plays a pivotal role in DG. We believe that any effective theory of grammar should have an explanation for how grammatical forms are acquired in both L1 and L2 contexts. We demonstrate that fluent language acquisition has a natural mechanism in DG, and that this mechanism

naturally explains the secondary role of Broca's area in comprehension. Additionally, the acquisition mechanism predicted by DG predicts the observed acquisition order described by language acquisition literature.

We also note that Decomposition Grammar is a theory of the *mechanisms* that produce and receive language, not a theory of the set of well-formed sentences in any given language. We argue that the set of sentences judged grammatical by speakers includes those that are well-formed under the Linear System, and that there are no formal constraints on how the Linear System interprets grammar other than: (a) it must process grammar sequentially from left-to-right, and (b) it uses the syntactic markers generated by the Tree System. Consequently, DG does not invoke the Performance-Competence distinction, which exists to reconcile divergence between what speakers produce and what they judge grammatical under the assumption that both should reflect a single underlying competence grammar. DG replaces that assumption with two mechanisms whose divergent structures directly predict the observed patterns. In other words, the Performance-Competence distinction exists to solve a problem that is not expressible in the language of DG.

The proposition of two systems at first seems improbable and unparsimonious. However, we argue that further inspection shows this theory to be much simpler than any theory that accepts only one system. We believe it has the following advantages:

1. Decomposition Grammar is based on a small set of primitive operations that can be derived from empirical constraints from neuroscience and cognitive science, alongside provable computational theorems based on those constraints.

2. We do not posit the existence of two grammars axiomatically; rather, the two grammars are a predictable consequence of primitive operations.

3. Decomposition Grammar itself is a simple system, but it predicts a large number of empirical results that have thus far resisted easy explanation. It provides a natural explanation for a wide variety of phenomena in aphasia studies, grammaticality studies, and language acquisition studies, without requiring any extra apparatus.

4. DG does not invoke some of the more complex machinery required by other grammars. It does not require transformation, syntactic movement, or any long-distance dependencies. All relationships are entirely local — specifically between left and right neighbours — with complex meanings being composed of these local relationships under recursion and iteration.

5. DG does not separate syntax from semantics at all. We argue that proposing two grammars adds no more theoretical complexity than proposing two different functions for syntax and semantics.

6. Our system is rigid and mechanical, and provides a large number of opportunities for falsification. We have not yet found any empirical evidence that Decomposition Grammar cannot explain without adding additional mechanisms to the theory.

3 Formal Model

Note: this section is terse and may seem unmotivated at times. Derivation and intuition follow after the core terms have been defined.

3.1 Primitive Objects

We define a space $\mathcal{C}$ of *semantic chunks*, denoted generically c. A semantic chunk is a semantically meaningful idea that a speaker can communicate. Some semantic chunks can be cleanly mapped to atomic lexical or phonological units — that is, expressed as individual words or morphemes without decomposition into a phrase. We call these *word-chunks*, and denote the space of word-chunks $\mathcal{W}$, with generic element w. Note that $\mathcal{W} \subset \mathcal{C}$. For modelling purposes we treat $\mathcal{C} = \mathbb{R}^n$; that is, we treat semantic chunks as n-dimensional vectors.

The *trace* $\tau(c)$ of a semantic chunk c is a property of the chunk representing a relational property overlapping with what other frameworks refer to as syntactic category. For example, the chunk c corresponding to the phrase "the cat" would have $\tau(c)$ corresponding to what is commonly called an NP in other frameworks. We note that this overlap is not coincidental — we argue that syntactic categories are a special case of the relational semantic properties encoded by traces, and that the distinction between syntax and semantics does not arise in DG. If a chunk does not have a trace we write $\tau(c) = \emptyset$, and say c has *null trace*.

A language has a fixed finite set of traces. Every trace is either *open* or *closed*, and a language has a non-zero number of both. Closed traces correspond to complete utterance forms — we call these *root traces*. For example, English has three root traces, corresponding to declarative, imperative, and interrogative sentences.

We additionally define a space $\mathcal{P}$ of *phonological forms*, with generic element p. A phonological form is a unit of phonological information with no attached syntax or semantics. It need not constitute a complete phonological form sufficient to produce a word — examples of complete phonological forms include the phonological

structure of the word "cat", while examples of incomplete phonological forms include Arabic triconsonantal roots or Yoruba tones. The use of the term "word" in "word-chunk" is therefore a convenience — a triconsonantal root is not a word, but it is a word-chunk.

To illustrate: the semantic meaning of "the dog chased the cat" is a semantic chunk in $\mathcal{C}$. "Cat" is a word-chunk in $\mathcal{W} \subset \mathcal{C}$.

The *Stack* is a memory resource accessible by both the Linear System and the Tree System. It is a Last-In, First-Out data structure storing semantic chunks from $\mathcal{C}$. It is highly limited in available memory — we denote its maximum length k, which we estimate to be a single-digit number.

3.2 Tree System Operations

The Tree System has two primitive operations: SPLIT and MERGE, which are inverses of each other. Both may only operate on the Stack.

SPLIT is a partial operation $\mathcal{C} \to \mathcal{C} \times \mathcal{C}$, defined $\text{SPLIT}_s(c) = (L, R)$. It takes a semantic chunk and returns an ordered pair of semantic chunks, determined by schema s. SPLIT is intimately connected to the MAP function, and in our model is never called without a MAP call. Chunks L and R are treated differently: R gets pushed to the Stack, while the algorithm recurses on chunk L. This will be described in more depth in the Split-Map Algorithm (Section 4.1).

There is a finite set of schemas $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$. Every call to SPLIT must use exactly one schema. For a given input chunk c, the schemas available are determined by $\tau(c)$ — for every trace τ, there is a nonempty subset $\mathcal{S}_\tau \subset \mathcal{S}$ of schemas that accept τ. There is at least one null schema $s_\emptyset$ that accepts chunks with $\tau(c) = \emptyset$.

The schema determines the traces of the output chunks: for all schemas s,

$$\text{SPLIT}_s(c) = (L, R) \quad \text{such that} \quad \tau(L) = \tau_{s,L} \quad \text{and} \quad \tau(R) = \tau_{s,R}$$

for any input chunk c. Intuitively, schemas should be understood as DG's analogue to production rules under a simple CFG model of language — the schema determines how the chunk is split and what the traces of the child chunks are.

SPLIT has a sub-process $\text{SEL}(\tau, c) \to \mathcal{S}$, which selects which schema to use when $\tau(c)$ admits multiple valid schemas. This function is not deterministic — the speaker actively chooses a schema based on semantic content and communicative intent.

MERGE is the inverse operation of SPLIT, $\mathcal{C} \times \mathcal{C} \to \mathcal{C}$. It operates on the Stack, taking the top two semantic chunks and merging them into one under a schema compatible with their traces. Unlike SPLIT, this is not guaranteed to succeed —

there is no guarantee that a schema exists compatible with a given pair of traces. When MERGE fails, it generates an error signal that is a crucial component of fluent language acquisition, as elaborated in Section *[ref to acquisition section]*.

3.3 Linear System Operations

The Linear System has access to the Stack as well as another memory resource called *State*, which is accessed by the function UPDATE. It additionally may perform a limited set of operations on the Stack.

State is a fixed-size resource utilised exclusively by the Linear System. It has no relationship to the Stack. It is a whole-brain resource that cannot be identified with any single brain region — it is a formal representation of active human cognition. The object stored by *State* is itself a semantic chunk in $\mathcal{C}$, and correspondingly has a trace. When we listen to someone speak and build a mental model of what they are saying, that mental model is *State*.

UPDATE is an operation $\mathcal{C} \times \mathcal{C} \to \mathcal{C}$. It is the central operation on *State*: it takes the current state chunk and another chunk, and returns an updated state chunk. When a speaker begins a sentence with "I saw a beautiful white fluffy…", *State* represents a mental model of seeing a beautiful white fluffy [something]. When the speaker completes the sentence with "car", UPDATE has updated *State* to represent a mental model of seeing a beautiful white fluffy car. Unlike SPLIT and MERGE, UPDATE does not have an inverse.

3.3.1 Linear System Stack Operations

The Linear System may interact with the top element of the Stack via the following operations:

- L-READ: The Linear System reads the top of the Stack. *[Note: implications for Update to be elaborated]*
- L-PUSH(c): The Linear System pushes chunk c to the top of the Stack.
- L-DELETE: The Linear System removes the top of the Stack.

3.4 Shared Operations

MAP and UNMAP are operations involved in mapping semantic chunks to phonological forms and back. We hypothesise they are handled by Wernicke's area, and may constitute its primary function. Since we claim the Stack is located in the inferior

frontal gyrus, we expect this interaction to be mediated by the arcuate fasciculus, which connects Wernicke's area and the inferior frontal gyrus. Unlike SPLIT and MERGE, MAP and UNMAP are not pure inverses, although we describe considerable shared functionality.

MAP is a partial operation $\mathcal{C} \to \mathcal{P}$, defined $\text{MAP}(c) = p$. It takes a semantic chunk and attempts to return a phonological form — that is, it succeeds only if c is a word-chunk. Upon success, c is removed from the Stack. Upon failure, $\text{SPLIT}(c)$ is called. As such, MAP is an integral part of the Split-Map algorithm. Note that when $\text{MAP}(c)$ is called, the Linear System briefly has access to c, regardless of whether or not a phonological form is successfully returned. This property will become important for our account of deliberate speech.

UNMAP is similar but not identical to the inverse of MAP, $\mathcal{P} \to \mathcal{C}$. It takes a phonological form and returns a word-chunk. If it successfully returns c such that $\text{MAP}(c) = p$, the Linear and Tree Systems both receive c. Specifically, $\text{UPDATE}(\textit{State}, c)$ is called, and c is pushed to the Stack.

If the phonological form does not correspond to a word-chunk, UNMAP returns $\emptyset$, the Stack is left unchanged, and *State* is not updated.

The *Semantic Delta* $\Delta_{a,b}$ denotes the difference between two semantic chunks, analogous to vector subtraction in machine learning:

$$\Delta_{a,b} = a - b, \quad a, b \in \mathbb{R}^n$$

Δ is defined $\mathcal{C} \times \mathcal{C} \to \mathcal{C}$; the output is a semantic chunk, though it need not be easily semantically interpretable. Our reasons for assuming we can compute a Semantic Delta are laid out in the section on the Unmap-Merge algorithm *[ref]*.

[Diagram of relationships between primitive operations to be inserted here]

3.5 Grammars and Languages

We denote the grammar of the Tree System G_T and the grammar of the Linear System G_L. The language generated by G_T is denoted $\mathcal{L}_T$, and the language accepted by G_L is denoted $\mathcal{L}_L$. We will demonstrate that G_T is a context-free grammar and $\mathcal{L}_T$ is a context-free language, while G_L and $\mathcal{L}_L$ are a context-sensitive grammar and context-sensitive language respectively.

3.6 Judged Grammaticality

We use the term *judged grammaticality* to refer to a listener's grammaticality judgements; it is thus a cognitive and behavioural property rather than a formal or computational one. However, we will show that judged grammaticality depends on the grammaticality of a sentence within the Tree System and Linear System.

Definition 1. *We define a four-valued judged grammaticality function $\mathcal{J}(s)$ for a string s, with values in* {CLOSED, OPEN, UNGRAMMATICAL, $\perp$}*, where:*

- CLOSED*: the utterance is both grammatical and complete.*
- OPEN*: the utterance is grammatical but incomplete — it could be continued to form a grammatical sentence.*
- UNGRAMMATICAL*: the utterance is ungrammatical regardless of any continuation.*
- $\perp$*: no grammaticality judgement is returned.*

We additionally define $\mathcal{J}_L(s)$ and $\mathcal{J}_T(s)$ as the grammaticality judgements of the Linear System and Tree System respectively, each taking values in the same set.

3.7 Types of Speech

We will refer to different patterns of speech throughout this paper. We define them as follows.

Definition 2 (Monitoring)**.** Monitoring *is the process of listening to one's own speech and interpreting it identically to how one would receive speech produced by others. Speech that involves monitoring is called* monitored speech*; if it is not monitored it is* unmonitored speech*.*

Definition 3 (Deliberate and Spontaneous Speech)**.** Deliberate speech *refers to production that involves cognitive processes outside the Split-Map algorithm used by the Tree System. Intuitively, it can be thought of as "thinking about what you say before you say it". It is typically co-occurrent with but nonetheless distinct from monitored speech. Speech that is not deliberate is referred to as* spontaneous speech*.*

Note that our use of the terms "deliberate" and "spontaneous" will exclusively refer to these definitions, which are specific to DG, and not to how these terms have been used elsewhere in other frameworks.

It must also be noted that deliberate speech does not imply a high degree of effort. In our model, all speech that involves any conscious thought after Split-Map begins

is deliberate speech. In practice, this may well mean that the great majority of observed speech is deliberate, while spontaneous speech is relegated to edge cases where a speaker speaks fluently while their conscious mind engages in other cognitive activities.

Definition 4 (Fluency)**.** Fluency *refers to the ability of the Tree System to produce a structure. That is, if a speaker can produce something in spontaneous speech, they can do so fluently.*

3.8 Memory Constraints

Following results in cognitive psychology, we assume that a speaker has a highly limited working memory with which they can process language. Miller's Law indicates that a human can hold 7 ± 2 discrete chunks of information in working memory at once.

We argue that this is an extremely tight constraint, and one that should, in principle, make language processing impossible. Communication frequently involves sentences of longer than 7 ± 2 lexical units. We note that results in cognitive psychology disagree over the precise number — we only require that the number of discrete information chunks we can hold in memory at once is some small number. We believe it sufficient to assume that $k < 10$.

It is therefore argued that our language faculties evolved to work around this constraint. However, language reception and language production are different computational tasks, and they require different data structures in order to achieve space-efficient algorithms.

We conjecture a formal limitation on the relationship between these algorithms:

Conjecture 1 (The Production-Reception Bound)**.** *The space complexity of the language reception algorithm must not exceed the space complexity of the language production algorithm.*

We consider this claim to be both intuitive and necessary — there is no research indicating a systematic asymmetry where produced speech is considerably longer than listeners are capable of understanding. However, there is no obvious reason to believe that the reception algorithm cannot have lower space complexity than the production algorithm: if speakers were capable of understanding sentences longer than those typically produced in fluent speech, this would not manifest as any impediment to communication.

This asymmetry forms the crux of Decomposition Grammar — that we require two radically different grammatical processing mechanisms to handle production

and reception, with the consequence that speech reception admits a superset of the grammatical sentences produced by fluent speech. We propose that fluent production follows a recursive algorithm we call Split-Map, whereas reception primarily uses one we term Unmap-Update.

3.9 Traces

The trace $\tau(c)$ of a chunk c is a generalisation of the role syntactic categories play in other theories of grammar. Traces are central to DG as they form the interface between the Linear System and the Tree System. We use the language of traces instead of syntactic categories for the following reasons:

1. Traces cannot be separated from relational semantic properties — they do not represent syntax separated from semantics.

2. The Linear System and Tree System have access to the same set of traces, but in radically different ways, of which only the Tree System's usage resembles the conventional syntax found in most frameworks.

The mechanics of DG are precise in how traces are used by our linguistic faculties. They are restricted by the following properties:

i. A language has a finite, non-empty set of traces, including at least one root trace.

ii. Every communicable semantic chunk has a trace.

iii. Both the Tree System and the Linear System have access to a semantic chunk's trace.

These properties of traces are sufficient for DG. That said, the process by which a speaker assigns a trace to a semantic chunk is, we argue, a special case of a more general cognitive capacity for ontological reinterpretation — the same capacity that allows us to reimagine a concrete object with altered properties, or to reframe a process as an object. We argue that it is no more within the scope of a linguistic theory to explain how a speaker can convert the semantic chunk corresponding to the property of 'red' to that corresponding to the abstract notion of 'redness' than it is to explain how a person can mentally convert the image of a green triangle to that of a red triangle.

In other words, we claim that human cognitive faculties allow us to perform complex operations on semantic objects; that this faculty is not merely linguistic in

nature; but that our linguistic faculties respond to a certain property of a certain subset of these operations — the trace property — in specific, algorithmic, and falsifiable ways.

We therefore blackbox the process of trace assignment entirely. We assume that the speaker may consciously produce a chunk of any desired trace with no limitations other than that the trace must exist and that the chunk has exactly one trace.

Lastly, we note that the terminology of "trace" is unusual for something that seems to heavily overlap with what other frameworks would simply call syntactic categories such as NP, VP, and so on. However, the process by which traces are used and generated is in many respects quite distinct from syntactic categories as conventionally understood, as will be discussed further in Section *[ref: Tree System in Acquisition]*.

With traces explained, we can begin building our model.

4 The Tree System in Production

4.1 The Split-Map Algorithm

We model spontaneous speech production as an algorithm involving the Split and Map functions being used in tandem. We call this algorithm *Split-Map*.

For a given target semantic chunk c^* with root trace, the speaker performs the following algorithm:

[Note: articulate clearly that outputting Map(c) reveals c to the Linear System — this is what the Semantic Delta is for]

Here SelectSchema is the speaker's learned behaviour of choosing from a set of valid schemas based on semantic information. When a phonological form is "outputted", we mean that it gets processed by the speaker's mechanism of converting phonological forms into audible speech, which we do not attempt to analyse.

Rephrased, this algorithm starts with a root semantic chunk — the idea that the speaker would like to express — and attempts to Map it to a phonological form. If Map returns null, the chunk is Split into left and right components. This forms a recursive algorithm performing Depth First Search on a tree structure. The speaker traverses from left to right across the tree. At each node c, the speaker uses Map to assess if c can be expressed as a lexical item. If it can, the speaker produces $\textsc{Map}(c)$ as speech. If not, c is forked by $\textsc{Split}(c) = (L, R)$: R is pushed to the Stack, and the algorithm recurses on L.

Algorithm 1 Split-Map

Require: Target semantic chunk c^* with root trace
Ensure: Sequence of phonological forms

```
procedure SplitMap(c)
    if Map(c) = w then
        Stack.remove(c)
        output w
    else
        s ← SelectSchema(c)
        (L, R) ← Split_s(c)
        Stack.push(R)
        SplitMap(L)
    end if
end procedure
Stack ← [c*]
while Stack ≠ [ ] do
    c ← Stack.pop()
    SplitMap(c)
end while
```

4.1.1 Complexity Properties of Split-Map

This algorithm has a number of important properties.

Time complexity. Split-Map has $O(n)$ time complexity, where n is the number of lexical items produced in output.

Space complexity. Split-Map has worst-case space complexity $O(n)$. This worst case is achieved when a sentence is entirely left-branching. Take, for example, the sentence "My sister's old cat returned". To produce this under Split-Map, five units of memory are required to be held at once — Split-Map must decompose the entire structure [[[[My sister's] old] cat] returned] before the first word "My" may be uttered.

The best-case space complexity is $O(1)$, observed with purely right-branching sentences such as [I [think [that [birds sing]]]], in which the leftmost word may be removed from the Stack as soon as it is produced.

This asymmetry is precisely why Split-Map must recurse on the left element: it allows the leftmost element of a sentence to be spoken and removed from the Stack immediately. A version of Split-Map that recursed on the right side would not have this $O(1)$ property for right-branching structures. That is, the fact that we speak from left to right means that right-recursion would require $O(n)$ space even in the best case.

This should mean that left-branching languages are effectively impossible under

the Tree System. We stand by this claim, and address it directly in Section *[ref: There Are No Left-Branching Languages]*.

[Note to be refined: We claim that deep-recursive left-branching structures do not exist in the Tree System. Right-to-left parsing in Unmap-Merge guarantees this, replacing left-branching structures with an alternative right-branching one. Shallow left-branching (e.g. the Japanese main verb in final position) is unaffected, but we predict that any left-branching feature capable of recursion to arbitrary depth comes with a word-order structure that permits reanalysis as right-branching under the Tree System — manifesting as topic-prominence in East Asian languages, and adjective ordering in English and German.]

4.1.2 Split-Map Generates a Context-Free Language

It can be seen that speech produced by Split-Map necessarily follows the structure of a Context Free Grammar. This is a consequence of Engelfriet's theorem.

Theorem 1 (Split-Map generates a CFL)**.** *The language produced by the Split-Map algorithm is a Context-Free Language.*

Proof Sketch. We observe that Split, as defined, generates a binary tree over semantic chunks. At each node, the production is determined entirely by the input chunk and its trace — specifically, by which schema $s \in \mathcal{S}_{\tau(c)}$ applies. The schema selection depends only on the local node and is independent of any surrounding context. This is precisely the defining property of a context-free production rule: the rewriting of a non-terminal is independent of its context.

The stack-based depth-first traversal of this tree by Split-Map reads off the leaves of the tree in left-to-right order. By the Engelfriet (1975) theorem — yield(RECOG) = CFL — the yield of any regular tree language is a context-free language, and every context-free language is the yield of some regular tree language. Since Split generates a regular tree language — each node is rewritten by a finite, context-free, local rule drawn from the finite schema set $\mathcal{S}$ — the left-to-right sequence produced by the stack traversal is necessarily a CFL. □

It must be emphasised that we do not *assume* that the language produced by Split-Map is a CFL, but *derive* it as a theorem from first principles.

4.1.3 Note on SelectSchema and Split

SelectSchema(c) and Split$_s(c)$ together constitute the core operation of Broca's area. SelectSchema selects a schema $s \in \mathcal{S}_{\tau(c)}$ based jointly on the trace and

semantic content of c — incorporating the speaker's communicative intentions, contextual factors, and the availability of lexical resources downstream. $\textsc{Split}_s$ then executes the decomposition, producing child chunks L and R with traces determined by s.

We treat SelectSchema as a formal sub-process of Split, and do not posit that it is used in any other linguistic process. We impose no further formal constraints on SelectSchema beyond the requirement that it return a schema $s \in \mathcal{S}_{\tau(c)}$. The specific mechanism by which a schema is selected given a chunk's trace and semantic content is a matter of cognitive implementation that we leave underspecified. We propose only that this joint SelectSchema–Split operation is the primary function of Broca's area, trained through the acquisition process described in Section *[ref: acquisition]*, and degraded in Broca's aphasia.

4.2 Note on Semantics and Syntax under Split-Map

Decomposition Grammar proposes that syntactic markers — for example affixes, grammatical tone, fused forms, and particles — are processed identically to semantic forms by the Split-Map algorithm, and that there is no functional difference between them.

The mechanism we propose is that during a split operation $\textsc{Split}(c) = (L, R)$, all necessary relevant information is passed down to the child nodes L and R via their traces $\tau(L)$ and $\tau(R)$. The child chunks thus have syntactic information encoded directly into them, which is then realised as phonological form when Map is successfully called. For an illustrative example, the semantic chunk representing "The dog chased the cat" would be split into chunks representing "The dog" and "chased the cat". The left chunk would then be split into "The" and "dog" — Map would then successfully return the phonological forms of each in turn.

It is therefore important to note that syntactic markers are, for all intents and purposes, normal word-chunks, and their representations in semantic chunk space are not functionally distinguished from other word-chunks.

4.3 Split-Map under Incomplete Acquisition

Not all language is produced from a position of complete acquisition. In an L2 context, for example, there is no reason to believe that every call to $\textsc{Split}(c)$ will successfully identify $\tau(c)$, or that, if it does, a valid schema exists with which to split c. We propose that when this happens, the speaker is consciously aware of their inability to split the form, and the Linear System intervenes. We will discuss

the precise mechanism of this in Section *[ref: Deliberate Speech]*.

5 The Linear System in Reception

We argue that the primary mechanism of language reception is linear, in contrast to the recursive stack produced by Split-Map.

5.1 The Inadequacy of CFG Parsing for Reception

We first demonstrate the inadequacy of true CFG parsing for speech reception under memory constraints. CYK, the standard CFG parsing algorithm, has $O(n^3)$ time complexity and $O(n^2)$ space complexity. Other CFG parsers have better average-case or best-case time complexity — algorithms such as Earley, GLR, and GLL have an average-case space complexity of $O(n)$. However, all of these are considerably more complex than the $O(\log n)$ space complexity of Split-Map. Thus, if speech is produced by Split-Map and the Production-Reception Bound holds, the reception algorithm cannot be any known true CFG parser.

We note that there is no proof that a CFG parser's average space complexity cannot be lower than $O(n)$. The known lower bound for CFG parsing space complexity is $O(\log n)$ [1], but no known algorithm has achieved better space complexity than $O(n)$.

We have further reasons to believe that the speech reception algorithm has better space complexity than $O(n)$ beyond the mismatch with Split-Map. Specifically, if speech reception's memory is handled by working memory, we would empirically see listeners having great difficulty processing sentences longer than approximately seven semantic units — which is not observed in practice.

We therefore propose that the primary mode of human language reception has no relationship to a recursive structure at all. We derive a linear reception algorithm as follows.

An intractable limitation of speech reception is that listeners can only receive one lexical item at a time — they cannot see the entire sentence at once, and therefore cannot unambiguously determine a sentence's grammatical structure until it is completed. This raises the possibility that the reception algorithm waits until all tokens are received before processing semantics. However, we believe this is highly unlikely. Garden path sentences such as "The horse raced past the barn fell" indicate that listeners build a semantic model of a sentence's meaning incrementally — they build a semantic interpretation from "The horse raced past…" before there

is enough information to unambiguously determine its structure.

We also believe that speech reception must have access to the same semantic chunk space $\mathcal{C}$ used by Split-Map. The alternative — that humans have two distinct and unrelated faculties for representing meaning in production and reception — would be extremely counterintuitive and create numerous problems for how production and reception match semantics. For instance, it would require that a speaker responding to "Do you live with your parents?" with "No, my parents live abroad" would be using two entirely different internal representations of "parents". We therefore propose that reception must utilise UNMAP, the inverse of MAP, as a means to translate received phonological forms into word-chunks.

We also observe that listeners meaningfully incorporate syntactic markers into reception. "He likes her" and "He liked her" are interpreted as having different semantics. By the property that syntactic markers are not distinguished from other word-chunks, we can see that the reception algorithm incorporates syntactic markers into building semantics, but not the generative process by which they were produced. We can state this more precisely as follows:

P1) Split-Map produces grammatical markers according to the rules of a CFG.

P2) The reception algorithm cannot be a CFG parser.

P3) The reception algorithm parses the grammatical markers produced by Split-Map.

Therefore:

C1) The reception algorithm parses grammatical markers in a manner distinct from the process by which they were generated.

This property is central to Decomposition Grammar, and will be used later to argue that receptive grammar admits a superset of spontaneous production.

5.2 The Unmap-Update Algorithm

Given the above properties, we define the *Unmap-Update* algorithm as follows:

This algorithm is straightforward. In natural language: the listener maintains a persistent state chunk c^* representing an internal semantic model of the speech received thus far. When a new phonological form p_i is received, it is converted to a word-chunk $w_i = \text{UNMAP}(p_i)$, and UPDATE incorporates w_i into the current state.

Since Unmap-Update only ever holds one internal state chunk, it has $O(1)$ space complexity, provided that the working memory requirements of a chunk do not

Algorithm 2 Unmap-Update

Require: Stream of phonological forms $p_1, p_2, \ldots, p_n$
Ensure: Semantic chunk representing the meaning of the utterance

$c^* \leftarrow \emptyset$
for each p_i in stream **do**
    $w_i \leftarrow \textsc{Unmap}(p_i)$
    $c^* \leftarrow \textsc{Update}(c^*, w_i)$
end for
return c^*

increase with its complexity. This is consistent with results from Schema theory *[citation]*, which show that the limits of working memory are invariant with the complexity of the chunks held in memory. We therefore take Unmap-Update to have $O(1)$ space complexity, conforming with the Production-Reception Bound.

5.3 The Semantic Oracle

This raises an immediate issue. An algorithm with $O(1)$ space complexity should not be capable of parsing a grammar stronger than a Regular Language, since a parser that holds a constant amount of information is a finite state machine. At first glance this seems like a fundamental problem for the Unmap-Update algorithm and for DG at large.

The resolution comes from a theoretical device standard in computer science. We know from the Halting Problem that no algorithm exists to determine whether a given Turing Machine halts — yet theoretical computer science routinely assumes the existence of such an algorithm, termed an *oracle*, in order to prove certain theorems. We adopt the same move here.

Definition 5 (Semantic Oracle)**.** *We assume the Linear System has access to a compression oracle $\mathcal{O}$ with the following property: let X be the set of all semantically interpretable meanings derivable from human language. Then $|\mathcal{O}(x)| = 1$ for all $x \in X$.*

In other words, the Oracle can compress arbitrarily rich semantic information into a single unit without ever requiring increased space. We are aware that this is likely a formal impossibility under standard information-theoretic constraints. However, we argue that it is a reasonable theoretical assumption given results from Chunking theory.

The literature on Chunking theory does not suggest any upper bound on the complexity or information density representable by a single chunk *[citations needed]*.

We have strict limits on how *many* chunks may be held in working memory at once, but a single chunk may be arbitrarily complex, limited only by semantic interpretability. A chess grandmaster can represent extremely rich information about a chess position in a single chunk that a novice cannot — not because the grandmaster has more working memory, but because they can interpret the position as a meaningful unit.

This raises a tension with standard assumptions about grammar. If we are to maintain that the grammaticality of a sentence is independent of its semantic interpretability — as in Chomsky's famous "Colourless green ideas sleep furiously" — our grammar must not differentiate between semantically interpretable and uninterpretable sentences. Yet we are claiming that semantic interpretability is precisely the property that allows the Linear System's grammar to be stronger than a Regular Language despite $O(1)$ space constraints.

Our solution is to define the Linear System's formal grammar, and likewise the oracle $\mathcal{O}$, as operating under the assumption of *perfect semantic interpretability*: we assume a listener with unbounded world knowledge, expertise, and intelligence, who can compress any theoretically chunkable information into a single chunk. We will later show that failure to achieve ideal semantic interpretability results in failure to arrive at a grammaticality judgement, rather than a judgement of ungrammaticality.

It must also be noted that the infinitely competent Oracle, while powerful, has a strict limitation. It can only store one semantic unit at a time and cannot track two or more competing semantic interpretations simultaneously, as this would violate the constant-information constraint. This forms the key limitation of the Linear System, which we call the *single interpretation principle*. It is this principle that strictly rules out true CFG parsers from our model, restricts the Linear System to a simple one-pass left-to-right parsing mechanism, and accounts for the property we term Update-Hardness, discussed in Section *[ref]*.

5.3.1 The Semantic Oracle and the Competence–Performance Distinction

There is some overlap between the Semantic Oracle and the Competence–Performance distinction. While Competence describes an idealised grammatical structure hidden behind the cognitive and personal complications of Performance, the Semantic Oracle describes an idealised perfect listener capable of correctly building a semantic model from any given utterance. Both devices exist, among other things, to allow the description of human language in terms of formal language classes — since formal languages are infinite objects but human linguistic capacity is finite, some

idealisation is required to make this description tractable.

The difference is this: Competence posits an idealised *structure* to be studied, while the Semantic Oracle posits an idealised *mechanism* as an abstraction of results in cognitive psychology.

We argue that the Semantic Oracle is therefore no heavier an assumption than Competence. We do not claim that perfect semantic interpretability is actually achievable — it is a theoretical tool used to derive the claim that the Linear System is context-sensitive, as shown in Section *[ref: Linear System as CSG Parser]*.

5.4 The Non-Invertibility of Update and Proposed Neural Pathways

An important feature of UPDATE is that, unlike SPLIT and MERGE, it is not invertible in general. Given a single chunk c^*, the Linear System has no mechanism for recovering two chunks c, w such that $\text{UPDATE}(c, w) = c^*$. There is a clear reason for this disparity: as outlined in Section *[ref: Unmap-Merge]*, the Tree System learns how to use schemas to split chunks via error signals provided by MERGE. UPDATE has no equivalent mechanism.

This non-invertibility implies that MAP and UNMAP cannot be strict inverses in the way that SPLIT and MERGE are. Specifically, UNMAP has two distinct functions: it provides word-chunks to MERGE during the Unmap-Merge algorithm (Section *[ref]*), and it provides word-chunks to UPDATE during Unmap-Update. In contrast, MAP is exclusively used in Split-Map and has no functional relationship to the Linear System.

This has implications for the neuroanatomical analysis of DG. Being non-strict inverses, we should expect MAP and UNMAP to involve a number of shared neural pathways, but with additional pathways used by UNMAP that are not used by MAP.

Neuroanatomy confirms the presence of structures compatible with this prediction. Broca's area and Wernicke's area are connected by the arcuate fasciculus, which can be divided into three primary segments: the anterior segment, the posterior segment, and the long segment. Aphasia studies provide revealing clues to the functional roles of each, as three distinct classes of aphasia correspond to damage to the three segments:

- Damage to the anterior segment is closely associated with Broca's aphasia.
- Damage to the posterior segment is closely associated with Wernicke's aphasia.
- Damage to the long segment is closely associated with Conduction aphasia.

This profile provides a direct analogue to functions within DG. Wernicke's aphasia is predicted to result from MAP failing to return a correct phonological form; we therefore predict that MAP utilises the posterior segment. Since UNMAP involves the inverse function of MAP in addition to other Linear System functionality, we propose that UNMAP utilises both the posterior and anterior segments. Chronic Broca's aphasia, associated with damage to the anterior segment or its separation from Broca's area, can then be modelled as failure to utilise UNMAP.

The role of the long segment will be modelled in a way that specifically accounts for the unusual profile of Conduction aphasia, discussed in Section *[ref: Conduction Aphasia]*. In brief, we argue that the long segment is responsible for pushing word-chunks to the Stack — handling the specific case where individual word-chunks enter the top of the Stack, as opposed to larger chunks pushed by the Split-Map algorithm.

6 The Tree System in Acquisition and Reception

We maintain that the Linear System is the primary mechanism of speech reception. However, this points at a theoretical weakness. If the Linear System is sufficient for language reception, and is not handled by Broca's area, why do patients with Broca's aphasia consistently display specific errors in reception? *[citation]* It follows that for DG to hold, the Tree System must be involved in reception to some degree. At first glance this seems undesirable for the theory — not only is it unparsimonious, we have also shown that tree-parsing is not feasible for reception.

We will show that this apparent discrepancy is not only consistent with the model, but necessary in order to build an account for fluent language acquisition.

6.1 Unmap-Merge in Acquisition

It is well documented in language acquisition literature that learning a lexical item is a separate process from acquiring it — that is, being able to use it fluently. Studies in language acquisition in both L1 and L2 contexts have consistently demonstrated that comprehension of both grammatical and lexical items develops before fluent production. We argue that this discrepancy can be understood as the Linear System acquiring the ability to UPDATE accurately before the Tree System can successfully SPLIT. Phrased differently, we identify fluent language acquisition with the building of schemas that produce correct traces.

This raises a question. An account for training the Linear System is straight-

forward, as the Linear System has access to *State* as a means to map tokens to semantics. The Tree System, as naively described, does not have such a clean mechanism. SPLIT requires schemas to produce chunks with correct traces — but how does it learn what the schemas are?

6.1.1 Semantic Delta and Predictive Coding

[Section to be expanded. Core claim: predictive coding describes a learning process computationally identical to backpropagation. The linear system provides a target, the tree system must recreate it, and the semantic delta is backpropagated through all merge steps to adjust parameters, building both schemas and SELECTSCHEMA*.]*

The Semantic Delta $\Delta_{a,b}$ provides a simple and effective explanation for how Split-Map can be trained, through using its inverse: Unmap-Merge.

For the purposes of deriving a semantic delta, semantic chunks are modelled as vectors in $\mathbb{R}^n$ following conventions in machine learning. As vectors, they have magnitude $|c|$. In isolation this does not obviously admit a meaningful interpretation in DG; however, $|\Delta_{a,b}| = |a - b|$ does — it is simply the size of the difference between two semantic chunks.

6.1.2 The Unmap-Merge Algorithm

Split-Map begins with the Stack containing one chunk. It pops the top of the Stack and attempts to MAP it. If MAP fails, it SPLITs the chunk and pushes the results to the Stack. It continues until the Stack is empty.

Unmap-Merge does the opposite. During reception, it receives a stream of phonological forms, UNMAPs them, and on each successful UNMAP pushes a word-chunk to the top of the Stack. It then merges pairs starting from the top, until only one chunk remains.

Note that this has the counterintuitive property of parsing in right-to-left order. Since UNMAP pushes the most recently received word-chunks to the top of the Stack, MERGE begins from the most recently retrieved chunks. Right-to-left parsing mechanisms are not typically hypothesised in theories of language reception, for the obvious reason that we hear language from left to right. Nonetheless, we claim that the parsing mechanism of the Tree System is a right-to-left algorithm. This is discussed in depth in Section 6.2.

For a stack of fixed length $[w_1, \ldots, w_n]$, we propose that schemas are trained as follows. Let $State_k$ refer to *State* after successfully parsing w_k, and let $\textsc{Merge}([w_{k+1}, \ldots, w_n])$ denote the chunk resulting from recursively running Unmap-Merge on the substack $[w_{k+1}, \ldots, w_n]$.

By the time MERGE has access to w_n, the Linear System has already parsed $w_1, \ldots, w_n$, and therefore $State_0$ is unavailable — *State* is now at $State_n$. Since Δ has the functional properties of subtraction, $State_0$ is recoverable as:

$$State_0 = \Delta_{State_n,\ \textsc{Merge}([w_1,\ldots,w_n])}$$

With $State_0$ recovered, we can generate the relevant targets for training MERGE by computing:

$$\Delta^* = \Delta_{\textsc{Merge}(State_0,\ \textsc{Merge}([w_1,\ldots,w_n])),\ State_n}$$

With Δ^* calculated, we have sufficient information to run the predictive-coding update mechanism *[elaborate]*.

Unmap-Merge must also have the property of preferring to fire when valid schemas are available for the two chunks being merged. Suppose we have a stack $[A, B, C]$ where the correct decomposition is $((A, B), C)$. Merging from right-to-left, Unmap-Merge will first attempt to merge $[A, [B, C]]$ — but if (B, C) do not merge under a valid schema, it will attempt $[[A, B], C]$ next.

Note that there are cases where $((A, B), C)$ is the correct decomposition but (B, C) admits a schema. Take the noun phrase "new wine taster", where the correct interpretation is "a taster of new wines". Unmap-Merge will first attempt to merge (*wine*, *taster*), and then (*new*, (*wine taster*)). The interpretation failure is only realised when comparing (*new* (*wine taster*)) to *State* — at which point Unmap-Merge must reverse two steps to arrive at ((*new wine*), *taster*). That is, if the magnitude of Δ between *State* and $\textsc{Merge}(State_0, ((new\ wine),\ taster))$ falls below a threshold z, the process restarts.

Unmap-Merge is therefore precisely the sort of multi-pass CFG parser with poor computational efficiency described in Section *[ref: Memory Constraints]*. These memory constraints greatly restrict the algorithm's scope and frequently cause failures, which we outline in Section *[ref: Merge-Hardness]*.

The Unmap-Merge algorithm for a fixed stack is as follows:
where z is a threshold determining when the match between $\textsc{Merge}(State_0, \textsc{Merge}(w_1, \ldots, w_n))$ and *State* is sufficiently close. Note that this algorithm applies only when the Stack is of fixed length. In most cases during reception the Stack is growing as new words are received; the full algorithm is outlined in Section *[ref: Unmap-Merge in Reception]*.

Algorithm 3 Unmap-Merge (fixed stack)

Require: Stack $[c_1, \ldots, c_n]$ of word-chunks

while *Stack*.size > 1 **do**
    counter $\leftarrow 0$
    while counter $+ 1 <$ *Stack*.size **do**
        if MERGE(*Stack*.top $-$ counter, *Stack*.top $-$ counter $- 1$) $\neq$ failure **then**
            MERGE(*Stack*.top $-$ counter, *Stack*.top $-$ counter $- 1$)
            break
        else
            counter $\leftarrow$ counter $+ 1$
        end if
    end while
    SPLIT(*Stack*.top)
end while
$State_0 \leftarrow \Delta(State,\ Stack.\text{top})$
$Merge\_state \leftarrow$ MERGE($State_0$, *Stack*.top)
if $|\Delta(State,\ Merge\_state)| < z$ **then**
    terminate
else
    ParseFailure
end if

6.2 Right-to-Left Parsing

We abbreviate left-to-right and right-to-left as L2R and R2L respectively throughout this section.

We claim that R2L parsing is not a formalism error, nor a point against DG's validity. Rather, it is a structural necessity.

6.2.1 Fixed Stack: Symmetry of L2R and R2L

For a fixed stack, L2R and R2L perform with identical worst-case space complexity over the set of all finite ordered trees. This follows trivially from the fact that if an L2R parsing algorithm is more efficient on some tree T, then the same algorithm implemented R2L achieves that same efficiency on the mirror image T^*.

6.2.2 Growing Stack: R2L Advantage

We lose this symmetric property when words are received in a continuous stream — that is, when the Stack grows during the parsing process. We term this a *growing stack*, and assume the best case where a parsing algorithm can perform any number of steps between words being pushed to the Stack.

We first demonstrate that R2L performs no worse than L2R on left-branching

structures, which represent the cases where L2R should be optimal. Consider the sentence "His brother's friend arrived", with tree structure [[[[*His*] *brother*'s] *friend*] *arrived*]. As each word is received, the growing stack proceeds as follows:

1. [*His*]
2. [*His*, *brother*'s] → [[*His*] *brother*'s]
3. [[*His*] *brother*'s, *friend*] → [[[*His*] *brother*'s] *friend*]
4. [[[[*His*] *brother*'s] *friend*], *arrived*]

With four lexical units, this requires a maximum of two elements in the Stack at any time. Crucially, an R2L algorithm on a growing stack follows the exact same procedure — because each new word arrives at the top of the Stack and can be immediately merged with the element below it. R2L performs identically to L2R on left-branching structures with a growing stack.

The property is not symmetric for right-branching structures. Consider [*the* [*cat* [*that* [*sat* [*on* [*the* [*m*
For both L2R and R2L, the tree cannot be parsed until the entire fragment is received.

However, an L2R algorithm must then traverse the completed stack from left to right to find valid merges. Even granting that it can immediately determine that [*the*, *cat*] do not form a valid merge, it must still traverse n elements — a provable time inefficiency. We further conjecture that an L2R algorithm on a growing stack will, in general, be forced to attempt merges that must subsequently be undone, incurring additional space costs. We do not prove this here, as it would require a space-optimal CFG parsing algorithm, which remains an open problem. The n-step traversal argument is intended as a lower bound illustration only.

R2L faces neither the provable time-complexity problem nor the conjectured space-efficiency problem on right-branching structures: it begins immediately with merging [*the*, *mat*], correctly.

6.2.3 R2L Guarantees Against Stack Overflow

We can derive a further advantage for R2L parsing. For any sequence s that can be correctly merged, R2L can do so without causing stack overflow. This guarantee does not hold for L2R.

Claim 1. *Let* $s = [s_1, \ldots, s_n]$ *where* $n - 1 > k$*, and suppose the correct parse is* $[s_1, [s_2, \ldots, [[s_m], s_{m+1}], \ldots, s_n] \ldots]$ *where* $m + 1 < k$*. That is,* s *is predominantly right-branching but contains one left-branching subcomponent* $[[s_m], s_{m+1}]$ *parseable*

without exceeding maximum stack length. An L2R parser will cause stack overflow on s; an R2L parser will not.

Proof Sketch. Since $m + 1 < k$, when $m + 1$ words have been received the L2R parser correctly merges $[s_m, s_{m+1}]$ as $[[s_m], s_{m+1}]$. However, since the remainder of the structure is right-branching, no further merges are possible until s_n is received, at which point the stack has reached length $n - 1 > k$, causing stack overflow.

R2L does not have this property. In the worst case where a correct parse requires more than k elements on the Stack simultaneously, stack overflow occurs before any incorrect merge is attempted. □

In other words, R2L will only attempt to merge a Stack that will not cause overflow, whereas L2R provides no such guarantee. This does not guarantee a successful parse — both R2L and L2R fail on any structure requiring more than k elements simultaneously. We term structures that exceed this limit *Merge-hard*, discussed in Section *[ref: Merge-Hardness]*.

6.3 Unmap-Merge in Reception

In the previous section we showed how Unmap-Merge is necessary for acquisition while maintaining Merge and Unmap as the exact inverses of Split and Map respectively. We now argue that this fully accounts for the Tree System's limited role in reception.

The Tree System acquisition mechanism requires Unmap-Merge to approximate *State* in order to function. We therefore propose a simple mechanism: when the Linear System fails to generate compatible semantics — specifically, when an incoming word-chunk's trace is incompatible with the trace of the current *State* — the Tree System's approximation of *State* is used as a fallback. This mechanism is not invoked in most reception, but serves as a backup for complex and ambiguous sentences.

The construction of Unmap-Merge makes clear that it will fail to fully parse whenever doing so requires the Stack to exceed maximum length k. In isolation this would make effective comprehension impossible, since many utterances require more than k tokens to reach grammatical completeness. We term such segments *Merge-hard*.

This must be contrasted with Unmap-Update's own failure points. Unlike Unmap-Merge, Unmap-Update has $O(1)$ space complexity and can in principle process utterances of arbitrary length. Its limitations are instead governed by the single interpretation principle. This is illustrated most clearly by garden path

sentences. Upon hearing "The horse raced", Unmap-Update updates *State* to a mental model of a horse racing. However, UPDATE is memoryless and irreversible — once the listener reaches "fell" and recognises that its trace is incompatible with the current *State*, comprehension fails, and there is no mechanism to retrieve and re-parse the sentence. We term such sentences *Update-hard*.

We claim that the backup comprehension mechanism provided by Unmap-Merge is invoked specifically for Update-hard sentences such as "The horse raced past the barn fell". This directly explains the specific comprehension deficits of Broca's aphasia *[elaborate]*: since Broca's patients lack the Tree System, they cannot invoke this fallback, and therefore fail on precisely those sentences that are Update-hard.

Furthermore, there is no reason to believe that sentences cannot be both Merge-hard and Update-hard simultaneously. We predict precisely the opposite: garden path sentences whose ambiguities extend beyond the maximum stack length k should be extremely difficult to interpret even for unimpaired individuals.

The research confirms this. Extended centre-embedded sentences such as "The mouse the cat the dog the child startled pursued ate the cheese" are almost impossible to interpret without sustained deliberate effort. Our account is straightforward: the Linear System's single *State* fails to assign interpretable semantics to "The mouse the cat the dog the child", and the Tree System fails to MERGE since the Stack overflows.

6.4 The Linear System as a CSG Parser

[Note: this section is currently the weakest and will be strengthened in revision.]

DG claims that the language of the Linear System is a superset of that of the Tree System. Specifically, we argue that the Linear System, given $O(1)$ space complexity assisted by the oracle $\mathcal{O}$, naturally gains the capacity to parse a Context Sensitive Language — a strict superset of the Context Free Language produced by the Tree System.

We strongly suspect that the Linear System's parsing algorithm uses logic similar to that of a dependency grammar (see Section *[ref: Metalinguistic Intuitions]*), but concede that the specific algorithm is an open problem outside the current model. Since DG claims that relationships are stored and tracked using $\mathcal{O}$, correctly modelling grammatical relationships is a question of whole-brain cognitive faculties far beyond the scope of the present work. However, DG can demonstrate that no parsing algorithm can be more space-efficient or time-efficient than the CSG-compatible system we propose.

6.4.1 Arbitrarily Many Dependencies

We assume that *State* is capable of tracking multiple dependencies within a single chunk. Consider the sentence: "The queen promised the knight who slayed the evil dragon that he could marry her daughter."

When the listener has received "The queen promised the knight who slayed", three relationships remain unresolved: "the queen" corresponds to "her daughter"; "the knight" corresponds to "he could marry"; and "slayed" to "the dragon". Nonetheless, the sentence is readily parseable. The listener successfully builds a single chunk representing something like "The queen promised the knight something in exchange for slaying something" — and when the sentence continues, these dependencies resolve via grammatical markers, word order, and semantic matching.

There is no obvious reason to fix the number of trackable dependencies at three, or at any finite number. We propose that *State* can track an arbitrary number of dependencies, provided that clear semantic interpretability is preserved throughout the left-to-right parse. Breaking semantic interpretability would require creating an additional chunk — which the Linear System, holding only one chunk at a time, cannot do. Failure to build a semantically coherent *State* therefore results in parse failure.

The limits of the Linear System's capacity to track multiple relationships simultaneously are thus a matter of semantics, not syntax. Under the assumption of ideal semantic interpretability (Section *[ref: Semantic Oracle]*), the Linear System can track arbitrarily many dependencies at once.

A language containing sentences with arbitrarily many simultaneous dependencies is necessarily a Context Sensitive Language, in contrast to a Context Free Language, in which sentences may track at most one unresolved dependency at a time *[check and cite]*. Under these assumptions, the language of the Linear System is a true CSL.

6.4.2 Optimality of the Linear System

We now demonstrate that no computational efficiency gains are available by restricting the Linear System to a weaker language class. By construction, the Linear System has $O(1)$ space complexity and $O(n)$ time complexity. No algorithm can improve on either bound: $O(1)$ space is strictly optimal, and $O(n)$ time is the minimum required to process every word in a sentence.

This shows that while the dependency grammar hypothesis of the Linear System cannot yet be rigorously proven, it is both sufficient and optimal. Proposing a

weaker language class cannot be justified on grounds of computational efficiency.

It also follows that this mechanism cleanly resolves the existence of grammatical non-CFG sentences without additional machinery. Structures such as Right Node Raising and Cross Serial Dependencies may be grammatical because they are context-sensitive and the Linear System accepts context-sensitive sentences. We do not require the apparatus of a Mildly Context Sensitive Grammar to explain their grammaticality. We do, however, require an account of how non-CFG structures are *produced* — since the Tree System cannot generate them in isolation. Their production via deliberate speech is addressed in Section *[ref: Deliberate Speech]*.

6.4.3 The Linear System Need Not Be "Too Strong"

Many theories of grammar expend considerable effort ensuring the grammatical system is not too powerful — strong enough to account for edge cases such as CSD, but not so strong as to permit structures never observed in human language. We argue this is not a problem for DG. The separate mechanism constraining such structures is stack memory: structures that break the $O(\log n)$ space property of the Tree System cannot be produced in deep recursive forms without causing stack overflow (see Section *[ref: Deliberate Speech]*). There is therefore no endogenous reason to argue that the Linear System's grammar need be weaker than full CSG.

6.4.4 On the Receptive Black Box

That reception follows a dependency grammar is not a core claim of Decomposition Grammar. The underspecification of the means by which semantic relationships are tracked by *State* is by design — it cannot be mechanistically determined from the core interactions of the Linear and Tree Systems alone. We wish only to demonstrate that a dependency grammar is computationally sufficient given verifiable properties of chunking theory. Whether the Oracle actually tracks relationships via a dependency grammar is a question DG is agnostic on, beyond the claim that the Oracle can track arbitrarily many relationships within a single chunk. The load-bearing mechanism is in chunking theory.

We also do not argue that the context-sensitivity of the Linear System must be complete context-sensitivity, or that mild context-sensitivity is impossible. We argue only that the standard reasons for proposing a mildly context-sensitive grammar do not apply within DG: such a restriction cannot be motivated by computational efficiency gains, which are impossible given the system is already optimal, nor by a need for a generative framework to produce non-CFG sentences, which DG argues is unnecessary.

There is considerable work to be done, independent of DG, on the specific nature of relationship tracking via whole-brain cognitive faculties. Many frameworks may be applied to this problem — dependency grammar, construction grammar, and others. If DG contributes to this literature, it does so by liberating grammatical theories from the unproven assumption that receptive grammar must mirror generative properties, and from the constraint that descriptive theories of reception must not be too powerful lest they accidentally permit structures too strong.

7 Schemas Are Not Syntax Rules

[This section is a working outline. Core intuition: the right-branching constraint forces the Tree System to follow rules very distinct from what the Linear System parses as grammatical, in the case of left-branching languages. For example, languages with prenominal adjectives have the Tree System schema adjective → adjective adjective alongside NP → adj NP. To be derived from Unmap-Merge and right-to-left parsing in the relevant section.]

We note here a point about the nature of simplicity in DG that will become important in what follows. The claim that DG is over-complex only applies if one's criterion of simplicity is a simple mathematical structure in which all grammatical structures fit neatly. This is not DG's notion of simplicity. DG's notion of simplicity is mechanistic: a complex structure produced by a simple mechanism is preferable to a simple structure that requires a complex mechanism to generate. This distinction is important and will recur throughout the remainder of the paper.

8 Meta-Linguistic Intuitions Are Probably Accurate for the Linear System

[Citations needed throughout; some claims need to be made more precise.]

The development of Universal Grammar and the introduction of the Performance-Competence distinction inoculated linguistics against taking meta-linguistic intuitions seriously — that is, how speakers report the subjective experience of interpreting grammar or deciding whether a sentence is grammatical. This was done for a clear reason: it was successfully shown that almost all observed language is generated by context-free rules, yet meta-linguistic intuitions virtually never reflected this. Before Chomsky, laypeople had not *[citation]* described the process of interpreting or producing grammar as one of composing trees under context-free rules. Rather,

they typically described it in terms of dependencies or other rules that bore no resemblance to the production rules that their grammaticality judgements clearly indicated they were following.

Under DG, this dismissal of meta-linguistic intuitions is unnecessary. DG provides a straightforward explanation for the mismatch between reported grammatical parsing mechanisms and observations of human speech: the context-free rules followed by the Tree System are inaccessible to conscious thought. When deliberate speech occurs, the speaker is intuitively following the context-sensitive rules of the Linear System. Meta-linguistic intuitions about grammatical parsing do not match context-free rules because listeners are literally not applying context-free rules — they are applying context-sensitive rules that admit a superset of what a CFG can produce.

This resolves an otherwise puzzling question. If meta-linguistic intuitions do not accurately report the process by which a listener arrives at a grammaticality judgement, why do we have them at all? Why would speakers report a rich and complex internal process for interpreting grammar that is in fact not used to arrive at the very grammatical judgements they claim to use it for? DG's answer is simple: the meta-linguistic intuitions are correct.

This provides our main clue towards the grammatical rules by which the Linear System parses received language, as discussed in Section 6.4.

9 Conduction Aphasia, the Long Segment, and Repetition

Conduction aphasia, which results from damage to the long segment of the arcuate fasciculus, has a highly unusual symptom profile that has thus far resisted clean explanation in neurolinguistics. Patients with conduction aphasia have largely unaffected comprehension and spontaneous production, but are severely impaired in repetition — that is, they have enormous difficulty repeating back speech they have just heard. An additional symptom is the inability to read sentences aloud.

This raises an immediate evolutionary question. The fitness purpose of verbatim speech repetition is dubious at best, and in any case insufficient to explain the evolution of a dedicated neuroanatomical structure. Furthermore, we could not have evolved a feature used primarily for reading prior to the invention of writing. This strongly indicates that the long segment serves a core linguistic function that is not directly visible in the surface profile of conduction aphasia symptoms, of which repetition and reading aloud are merely sub-processes.

We therefore seek to identify a function of the long segment satisfying the following properties:

1. It has a clear and necessary function for language.
2. Its absence results in repetition failure and inability to read aloud.
3. It is not necessary for spontaneous production or comprehension.

The combination of properties 1 and 3 seems almost paradoxical — production and comprehension appear at first glance to constitute the entirety of necessary language functions.

The centrality of acquisition to DG provides a resolution. We hypothesise that the long segment has a crucial role in language acquisition, and that the faculty of repetition is a side effect of this primary acquisition function.

9.1 The Long Segment as Stack-Push Pathway

There is a core feature of Unmap that is used exclusively in the Unmap-Merge algorithm: the pushing of word-chunks to the Stack. The other two algorithms — Split-Map and Unmap-Update — never require this. Split-Map checks whether the left-side chunk Maps before pushing anything to the Stack, so a word-chunk, which by definition maps successfully, never enters the Stack via Split-Map. Unmap-Update does not operate on the Stack at all.

We identify this Stack-push functionality with the long segment. Of the three primary segments of the arcuate fasciculus, two are already accounted for: the anterior segment is associated with the Unmap-Update algorithm, allowing word-chunks to update *State*; the posterior segment is associated with the core Map/Unmap functionality, accounting for Wernicke's aphasia *[Note: the precise division between anterior and posterior segment functions needs further refinement to cleanly account for Wernicke's comprehension deficits. Working hypothesis: the posterior segment handles shared Map/Unmap core functionality; the anterior segment handles the additional Update-specific pathway.]*

Since Unmap-Merge is necessary for the acquisition of fluent spontaneous speech, the features absent in conduction aphasia are those which are side effects of this core acquisition function. We model speech repetition as follows.

During listening, word-chunks are pushed to the Stack and merged via Unmap-Merge. This results in the Stack containing one or more merged chunks at varying levels of completeness *[Check: does this necessarily produce a weakly size-increasing*

stack, with smaller chunks at the top and larger ones towards the bottom? If so, is this important for the model?]. The speaker can then simply run Split-Map on this filled Stack, producing a fluent reproduction of the received speech. Repetition is thus a straightforward application of functions already required for acquisition, not an independent faculty.

Failure to read aloud follows from the same mechanism: reading aloud requires pushing word-chunks to the Stack for production, which is precisely the function lost in conduction aphasia. This explains why reading comprehension is preserved in conduction aphasia while reading aloud is not.

9.2 Consistency with Update Non-Invertibility

This account is consistent with the non-invertibility of UPDATE. Since UPDATE cannot be reversed, the Linear System provides no mechanism for accurately reproducing received speech. It does provide a mechanism for reproducing the *meaning* of an utterance — via paraphrasing. The listener builds *State* via Unmap-Update, then uses that chunk as a root chunk for Split-Map, with no guarantee that the output will match the structure of the received sentence. This is consistent with the observation that conduction aphasia patients' ability to paraphrase is preserved [5].

9.3 Predictions for Conduction Aphasia

Since both Broca's aphasia and conduction aphasia involve failure to run Unmap-Merge, and Broca's aphasia reception failures are explained by this failure, we predict that conduction aphasia patients should show comprehension failures with considerable overlap with those of Broca's aphasia patients.

The evidence confirms this. Conduction aphasia patients do display the same specific and often subtle comprehension failures seen in Broca's aphasia patients [6]. *[Investigate subtle differences between the two profiles and determine whether these fall out of the model.]*

This account makes a further crucial prediction: conduction aphasia patients should lose the capacity for fluent language acquisition — specifically, the ability to acquire fluent spontaneous production — while retaining the ability to comprehend grammatical structures and acquire receptive competence. To our knowledge, no study of L2 acquisition profiles in conduction aphasia patients has yet been conducted. DG strongly predicts that such a study would find significantly impaired fluent L2 acquisition relative to control, with receptive acquisition remaining intact.

A corollary prediction concerns unimpaired individuals: the ability to repeat

speech verbatim should be positively correlated with the degree to which the relevant structures have been acquired. An L2 speaker should struggle to repeat verbatim structures they cannot yet produce fluently in spontaneous speech, but repeat successfully structures they have fully acquired. *[Has this been confirmed in the literature?]*

10 Acquisition and Acquisition Order

We claim that fluent language acquisition can be identified with the Tree System acquisition process. The following statements are equivalent in DG:

1. A speaker has acquired structure X.
2. The Tree System has learned a schema corresponding to structure X.
3. A speaker can produce structure X in spontaneous speech.

As an important corollary, the following statements are also equivalent in DG:

1. Structure X is produceable under the rules of a context-free grammar.
2. Structure X can be acquired.
3. Structure X can be produced in spontaneous speech.

These equivalences have considerable predictive implications. Their consequences for non-CFG structures will be discussed in Section *[ref]*. We now discuss the implications for acquisition order theory.

Claim 2. *The order in which grammatical features are acquired is the order in which they can form complete utterances when parsed in right-to-left order.*

11 There Are No Left-Branching Languages (For the Tree System)

That DG strongly predicts against left-branching languages should, at first glance, make the theory untenable. This was precisely the objection *[cite]* raised against Yngve *[cite]*, who similarly proposed a stack-based model of production based on working memory constraints. Yngve's theory was based primarily on English, and he argued that the right-branching preference of English could be explained by the

need to limit space on the Stack — however, his model did not have a clear answer for how speakers of heavily left-branching languages such as Japanese could avoid this problem.

At first glance, DG seems to suffer from the same issue. Formally, it is true that left-branching production rules are incompatible with the Tree System. However, we will demonstrate how the acquisition mechanism of Unmap-Merge, when trained on a left-branching grammar, creates right-branching production rules — and how this directly and mechanistically explains observed typological properties in human languages, namely that left-branching languages are tightly correlated with topic-prominence and other word-ordering preferences that right-branching languages tend not to have.

12 Pragmatics and Word Order

[This section will need a rewrite for clarity once the core argument is stabilised.]

We observe that a number of thoroughly-studied languages, such as Japanese and Mandarin, have heavily preferred word-ordering rules that do not neatly align with grammaticality as commonly studied.

We begin with Japanese as a canonical case. Traditional analyses describe Japanese grammar as unusually left-branching, with a final main verb and exclusively prenominal modifiers. It additionally contains a sentence-level topic marker, は (*wa*), and Japanese sentences are often analysed at a Topic-Comment level. These form strict grammatical rules, and failure to observe them — for example, placing the verb before the object — results in ungrammaticality.

Yet Japanese also follows strict pragmatic word-ordering rules. Consider the sentence:

明日、学校で先生にプレゼントを上げます

meaning "[I'm] going to give a present to [my] teacher at school tomorrow", of the form *Tomorrow, at school, to the teacher, a present, give.* Reordering this as 先生にプレゼントを学校で明日あげます — *To the teacher, a present, at school, tomorrow, give* — preserves meaning, grammaticality, and comprehensibility entirely, since the grammatical role of each element is marked by particles *[source]*.

Both linguists *[check]* and Japanese learning materials describe the preferred order as *big-to-small*: starting with the broadest frame (tomorrow), then narrowing progressively — place, then object, then verb.

This account is insufficient. In what sense is "tomorrow" a bigger frame than "at school"? On what axis is this measured? The big-to-small account seems to propose an almost metaphysical property — that time is in some sense larger than space — which is not self-evidently false but is not a satisfying scientific explanation.

A simpler retort might be that time is merely treated as larger than space as a contingent pragmatic convention specific to Japanese. But this fails to explain a much larger issue: the same ordering is widely documented across SOV languages with no clear linguistic or cultural connection to Japanese, including Tamil, Turkish, German, and Afrikaans. This is documented as the Time-Manner-Place ordering *[citation]*. We do not believe claims of language contact or shared cultural inheritance will withstand scrutiny here.

This phenomenon appears intimately connected to the reverse ordering — Place-Manner-Time — observed in some SVO languages such as English and French. An English speaker is more likely to say "She drove to work slowly this morning" than "She drove this morning to work slowly". Importantly, not all SVO languages follow this pattern: Mandarin strongly prefers Time-Manner-Place despite being SVO.

A parsimonious theory of language should account for:

1. The existence of pragmatics — word-ordering preferences that do not affect comprehensibility, meaning, or grammaticality, but are nonetheless widely observed by native speakers.

2. The close connection between SOV typology and the Time-Manner-Place pragmatic order.

3. The asymmetric relationship between SVO typology and the Place-Manner-Time order — present in English but not in Mandarin.

We demonstrate that all three are directly predicted by DG, drawing on empirical research in developmental psychology.

12.1 Acquisition of Pragmatics under Left-Branching Structures

We demonstrate that the Tree System acquisition mechanism predicts that a learner acquiring a left-branching language will build schemas corresponding to pragmatic relationships, either alongside or in place of traditionally understood grammatical relationships.

Suppose a learner of a heavily left-branching language X has acquired sentences of the form $[A, B]$ — that is, they have acquired a schema a such that $\textsc{Split}_a(c) =$

(A, B) for chunks c with trace $\tau_{A,B}$, with the left-branching property that the correct merge is of the form $[[A], B]$. However, they have not yet acquired a schema z such that $\textsc{Split}_z(c) = (Z, Y)$ where $\tau(Y) = \tau_{A,B}$, and Z corresponds to a new grammatical structure for which they have no schema at all. In other words, they can $\textsc{Merge}(A, B) = Y$, but cannot yet merge structures of the form $[[[Z]\ A]\ B]$ from sentences of the form $[Z, A, B]$, since doing so would require first acquiring $\textsc{Merge}([[Z]\ A])$, which by construction they cannot yet do.

Given R2L parsing and the already-acquired ability to merge $[A, B]$ as $[[A], B]$, the learner will merge in the following order:

$$[Z, A, B] \ \rightarrow\ [Z,\ [[A]\ B]] \ \rightarrow\ [Z\ [[A]\ B]]$$

Since X is formally left-branching, the merge $[Z\ [[A]\ B]]$ does not respect X's formal syntax. However, this is irrelevant to the acquisition mechanism: if $[Z\ [[A]\ B]]$ correctly constructs semantics matching *State*, the schema $\textsc{Split}_c(Y) = (Z, Y)$ will still be trained.

Even though the language is grammatically left-branching, the structure acquired by the Tree System is nonetheless right-branching.

This creates a superficially paradoxical situation. The learner acquires the ability to build and decompose semantic chunks via a rule that bears no resemblance to the grammatical-semantic rule by which those chunks are related in *State*. If the learner's *State* represents the semantics of "The dog chased the cat" expressed in the left-branching grammar of X — i.e. $[[[dog]\ cat]\ chased]$ — the Tree System learns to split these relationships in the form $[dog\ [[cat]\ chased]]$ instead. What, then, is the semantic relationship under which $[dog\ [[cat]\ chased]]$ is split, if it has no relationship to the grammatical-semantic structure embedded in *State*?

We propose that this relationship is *discursive* rather than semantic-relational — and specifically, that it is identical to a topic-comment relationship. Pragmatic word order and topic-comment structures are therefore two manifestations of the same underlying process.

12.2 Pragmatic Word Order and Topic-Comment Reflect the Same Property

We denote the discursive relationship $\triangleright$ and call it *frames*. $A \triangleright B$ is read as "A frames B".

We conjecture that $\triangleright$ is the simplest relationship expressible when other semantic-grammatical relationships are unavailable: $A \triangleright B$ means that B "says something

about" A, with no commitment to what exactly it says. This essentially rephrases how topic markers are commonly explained — the Japanese structure A は B is often rendered as "As for A, B". However, the "as for" framing implies a contrastive element not fundamental to $\triangleright$. Sentences with は frequently imply not only $A \triangleright B$ but that $C \not\triangleright B$ for some alternative topic C — as in "As for me, I like baseball (as opposed to someone else who does not)" *[cite]*.

In heavily left-branching languages, $\triangleright$ is one of the primary relationships used by the Tree System to produce right-branching structures. A left-branching grammatical analysis of [*Tomorrow, at school, to the teacher, a present, give*] would yield [[[[[*Tomorrow*] *at school*] *to the teacher*] *a present*] *give*], but the Tree System produces it as:

$$[Tomorrow \triangleright [at\ school \triangleright [to\ the\ teacher \triangleright [a\ present \triangleright [give]]]]]$$

We therefore claim that the relationship between left-branching languages and topic-prominence is considerably deeper than is often supposed. What have been called pragmatics and topic-prominence are two descriptions of the same core operation $\triangleright$. Japanese, for instance, is often described as topic-prominent in the sense that a sentence may have one or two topics marked by は *[cite]*. DG claims that a sentence may have an arbitrary number of $\triangleright$-topics, most of which are never particle-marked; when topic *is* marked by は, it introduces the contrastive element that the unmarked $\triangleright$ relation lacks.

To avoid confusion with existing analyses, we adopt the following terminology. The non-contrastive topic-comment relationship represented by $\triangleright$ will be called $\triangleright$-*frames* and $\triangleright$-*comments*. The contrastive topic-comment relationship will be called *contrast-topics* and *contrast-comments*.

12.3 $\triangleright$-Frames Predict Typological Relationships

Since topic-prominence was first described as a typological property, its correlation with left-branching (head-final) typology has been widely observed *[Li & Thompson 1976, cite properly]*, but no widely agreed-upon mechanistic explanation has emerged. Left-branching languages from widely different areas and cultures — Mandarin, Turkish, Basque, Hungarian — all show a relationship with topic-prominence.

DG provides a mechanistic account without additional stipulations. When a learner's Tree System acquires on a grammatically left-branching language, it will acquire right-branching $\triangleright$-frames, of which topic-prominence is one description. We argue that this explanation of a long-mysterious typological property is a key

empirical strength of DG.

12.4 Case Study: Mandarin Chinese

[Syntax trees to be added; find additional examples if possible.]

Mandarin is a language for which the two-system analysis becomes particularly striking. Although it is an SVO language, nearly every other aspect of Mandarin grammar is deeply left-branching: noun modifiers are prenominal, and all prepositional phrases precede the verb *[find standard linguistic framing for this]*. By the naive interpretation of Split-Map, spontaneous Mandarin production should rapidly exceed Stack length and be effectively impossible. Yet Mandarin is also an extremely topic-prominent language, with many linguists arguing that topic-comment is its fundamental grammatical structure.

Consider the following sentence:

我	昨天	在	超市		的	西	全	都	坏	掉	了
wǒ	zuótiān	zài	chāoshì	mǎi	de	dōngxi	quán	dōu	huài	diào	le
1SG	yesterday	at	supermarket	buy	REL	thing(s)	all	all	bad/spoil	RVC	PFV

That is: "All the stuff I bought at the supermarket yesterday has gone bad."

A standard grammatical analysis under a binary syntax tree, accepting left-branching and without invoking topic-prominence, yields a structure of the form:

[我 [[[[昨天] 在超市] 的] 西] 全都] [坏掉] 了]]

[Check and add syntax tree diagram.]

Under this analysis, the entire sentence after 我 must be split before the first prepositional phrase 昨天 is uttered, which would cause Stack overflow.

The ▷-frame structure resolves this. We propose a right-branching analysis:

[我 ▷ [昨天 ▷ [在超市 ▷ [的西 ▷ [全部 ▷ [坏掉了]]]]]]

Intuitively: *Me* ▷ [*yesterday* ▷ [*at the supermarket* ▷ [*the stuff I bought there* ▷ [*all of it* ▷ [*gone bad now*]]]]].

This is the essence of the relationship DG posits between left-branching and topic-prominence: heavily left-branching grammatical structures necessitate the use of word order to organise information efficiently, and this manifests as ▷-framing becoming the primary schema the Tree System uses to generate CFG trees during spontaneous production.

We can show how the ▷-frame schema is trained during acquisition. An individual acquiring Mandarin learns on heavily left-branching structures such as 我今天早上游泳 — "I went swimming this morning" — represented in a left-branching tree as [我 [[[今天] 早上] 游泳]]. The R2L property of Unmap-Merge first attempts to merge (早上, 游泳). The "correct" left-branching merge would only be attempted if the preceding R2L merges fail. R2L therefore strongly incentivises ▷ to be learned before left-branching syntactic relationships are acquired. This is the mechanism by which left-branching languages produce topic-comment structures — both as a statistically learned preference and as a means of producing spontaneous speech without Stack overflow.

In short: ▷-frames convert enough left-daughters into right-daughters for spontaneous production to remain within Stack bounds.

12.5 Time-Manner-Place as the Reverse of Developmental Concept Acquisition Order

[This section requires further development and engagement with specific languages. Treat as a working outline.]

The regularity of the Time-Manner-Place pragmatic order across unrelated SOV languages poses a challenge for any theory of acquisition. There is nothing obviously special about the semantics or syntax of this specific ordering. Meta-linguistic intuitions about time being a "larger" scope than place require speculating about deep semantic properties of time and space, which is far from appropriate for a theory of language acquisition. Yet if Time-Manner-Place is not special, we are at a loss to explain why so many unrelated languages converge on it.

The mechanism described in the previous section provides a clear explanation without additional stipulations, via observations from developmental psychology.

For Tree System acquisition to function, the Semantic Delta must generate reliable error signals by comparing $\textsc{Merge}(State_0, \textsc{Merge}(w_1, \ldots, w_n))$ with $State_n$ (Section *[ref: Unmap-Merge in Acquisition]*). This requires that the semantics of both sides be interpretable — if the learner cannot understand what is being said, the acquisition mechanism cannot fire.

Young children famously acquire the ability to express concepts in a specific developmental order *[find better source than lispeech.com]*. The sequence runs approximately: objects, events, actions, adjectives, adverbs (manner), spatial concepts, temporal concepts.

We note the partial correspondence between this developmental order and the

Time-Manner-Place ordering: temporal concepts map to time, spatial concepts partially to place, adverbs partially to manner.

We therefore propose that the Time-Manner-Place order is determined by the *reverse* of the childhood concept acquisition order. We expand this claim beyond Time-Manner-Place: the pragmatic word-order preference of $\triangleright$-frames in left-branching languages is generally determined by the reverse of childhood concept acquisition order, extending to object phrases, adverbs, and other flexibly placed constituents.

12.5.1 Demonstration: Word Order Preference from Developmental Order

Suppose a learner of language X can understand spatial concepts but not yet temporal ones, and is receiving input from fluent speakers conforming to X's canonical word order. The learner receives sentences of the forms [Time-PP, VP], [Time-PP, Place-PP, VP], and [Place-PP, VP], but not [Place-PP, Time-PP, VP], as this violates canonical order.

Of these three input types, two involve temporal concepts, which the learner cannot yet interpret. The learner therefore acquires the schema to split and merge [Place-PP, VP] first.

Later, when the learner has acquired temporal concepts but has partially or fully acquired the [Place-PP, VP] schema, they begin receiving all three sentence types successfully. They now acquire [Time-PP, VP] and [Time-PP, Place-PP, VP] schemas — but since [Place-PP, VP] was acquired first, the Tree System has already embedded Place as the more deeply learned right-branching constituent. Time, arriving later in acquisition, frames Place from the outside.

This predicts that [Place-PP, Time-PP, VP] should not be a preferred order in SOV languages: it would require Time to be the more deeply embedded constituent, which contradicts the developmental acquisition order. *[Develop this argument further with specific language data.]*

12.6 The Opacity of Topic-Prominence

Another observed feature of many topic-prominent languages is that despite topic-prominence being a thoroughly vetted linguistic feature, it rarely appears in intuitive explanations of grammar outside of the field of linguistics. In L2 Mandarin teaching, for example, topic-prominence rarely if ever appears early on as a fundamental grammatical rule for learners, although it is occasionally raised by heterodox teachers

[citation — investigate whether this is citable].

This seems to relate to other trends observed in topic-prominent languages — that they tend to have the dual-structure discussed earlier *[write this properly]*, and that while topic-prominence is central, it seems to closely resemble pragmatics in its irrelevance to grammaticality.

This can be explained by $\triangleright$-framing being a discursive rather than a semantic-grammatical relationship. By construction, while it is used to construct *State*, it does not reflect any of the relational properties embedded inside it. Thus, when reflecting on *State* to build meta-linguistic intuitions, $\triangleright$-framing does not appear prominently. It is resistant to the self-reflection that reveals other Linear System grammatical properties. Meta-linguistic intuitions are therefore more likely to reveal information that can be semantically embedded — nouns, verbs, subjects, objects, cases, tense, and so on.

12.7 Explicit Topic Marking

[To be developed. Core claim: Japanese and Korean have explicit topic marking, but this is a red herring — it allows speakers to consciously reflect on topic marking without noticing the $\triangleright$-frame structure operating recursively beneath it.]

13 Grammaticality

Assessing grammaticality is something humans can do. It does not play a central role in DG — we posit two different grammatical parsing mechanisms that a listener cannot easily differentiate, and a grammaticality judgement does not tell us which of the two systems assessed the sentence as grammatical. Grammaticality judgements are thus of limited use to DG if we wish to delineate each system's grammar. However, DG does require an explanation of where grammaticality judgements come from.

When Broca's patients are given Broca-hard sentences, they typically fail to derive a correct parse, or report that they cannot assess the sentence's grammaticality at all. For DG, this indicates that parse failure does not return UNGRAMMATICAL so much as it fails to return any judgement — that is, it returns $\perp$. However, many sentences that are presumably Broca-easy can be immediately recognised as ungrammatical, for example "I is bald swede".

It is also worth delineating a further form of grammatical failure. There are sentences that are ungrammatical because they strictly violate rules in a way that can

be immediately identified without further listening. These contrast with incomplete sentences such as "I went to the", which could be grammatical if continued, but if presented as a complete utterance, are ungrammatical.

The four values of $\mathcal{J}$ defined earlier — CLOSED, OPEN, UNGRAMMATICAL, and $\perp$ — are therefore exactly what is required. The value returned must derive from how the two underlying systems parse the utterance.

Since the Linear System admits a superset of what the Tree System can produce or parse, our account simplifies as follows. When the Tree System provides a judgement on utterance s other than $\perp$:

$$\mathcal{J}(s) = \mathcal{J}_L(s)$$

When $\mathcal{J}_L(s) = \perp$:

$$\mathcal{J}(s) = \mathcal{J}_T(s)$$

14 Transformation and Syntactic Movement

DG does not posit the existence of any transformation or syntactic movement. We claim that the linguistic phenomena that syntactic movement describes are already covered by other aspects of the model.

We use wh-movement as the canonical example. Many frameworks require wh-movement to explain the relationship between sentences like "John ate a cat" and "Which cat did John eat?" We do not believe this is necessary. Specifically, we argue that interrogative, declarative, and imperative sentences each correspond to different root traces, and the relationship between them falls out of this distinction without any movement operation.

Additionally, the syntactic movement required by other frameworks to explain long-distance dependencies is not required by DG, as these are covered by the interaction of the simultaneous left-to-right parsing of the Linear System and right-to-left parsing of the Tree System in comprehension.

15 Two Grammars

The claim that there are two grammars may be the most provocative result of DG. However, it falls out cleanly from the model as built.

$\mathcal{L}_L$ meets the formal definition of a language in that there are sentences it can contain or cannot contain, as determined by the grammar $\mathcal{G}_L$. Similarly, the

sentences in $\mathcal{L}_T$ that the Tree System can both parse and produce are constrained by context-free rules, and therefore $\mathcal{G}_T$ is also a grammar. We have shown that $\mathcal{L}_L$ is context-sensitive while $\mathcal{L}_T$ is context-free. Since a context-free language cannot equal a context-sensitive language, and a context-free grammar cannot equal a context-sensitive grammar, it follows that there are two grammars.

In other words, the claim that there are two grammars is no less likely than the model of DG itself — if there are two systems with the properties described, there must be two grammars.

16 Map and Unmap as Wernicke's Area Functions

The identification of Broca's area with Split and Merge is one of DG's central claims. Since Map and Unmap are used by both systems, our identification of them with Wernicke's area is neither novel to DG nor a central claim. However, much existing research already indicates that mapping semantics to phonological forms is a core function of Wernicke's area *[citations]*. The relevance to DG is twofold:

1. It allows us to propose Map and Unmap as functions without adding theoretical machinery with no empirical basis.

2. It allows us to make specific theoretical predictions about Wernicke's aphasia.

Specifically, recall that Split-Map terminates on a given node when Map returns a valid phonological form. This predicts that when Map fails to return a valid phonological form — as in Wernicke's aphasia — Split-Map does not terminate. This predicts that patients with Wernicke's aphasia should speak in longer sentences than unimpaired individuals, with sentence length increasing with symptom severity.

This is exactly what the data shows *[[4]; additional citations needed]* — studies have repeatedly shown that Wernicke's patients speak in markedly longer sentences than unimpaired individuals.

As a corollary, patients with milder Wernicke's symptoms — for whom word retrieval is impaired but not absent — should show a pattern of increased circumlocution, fluently substituting a phrase where an unimpaired speaker would produce a single word. When Map fails to return a normally chosen phonological form, the chunk is Split again, producing a longer but fluent elaboration. The data is consistent with this prediction *[citation]*.

17 Interlanguage Fossilisation

[Unfinished — the following presents the root idea only.]

We argue that fossilisation — in particular, the fossilisation of lexical, grammatical, and pragmatic errors, as opposed to phonological errors — occurs when an L2 learner's Tree System acquisition runs on the learner's own output, or in some cases on input from other non-native speakers. This presents a clean and coherent account of several phenomena associated with interlanguage fossilisation: how it is worsened by forcing early output, how fossilised errors cannot be unlearned in fluent spontaneous speech, and yet how the same speaker can readily notice and identify the error in received speech. The last of these follows directly from the fact that the Linear System, which handles reception, is not impaired by the fossilised Tree System schemas.

18 The Incompatibility Argument

[This argument may be included in a revised version. It requires further elaboration, and the case for preferring invertibility needs to be made explicitly.]

Claim 3. *A theory of grammar cannot simultaneously have all of the following properties:*

1. *Production and reception are inverses of the same functionality.*
2. *The theory accounts for the asymmetry between Broca's aphasia's comprehension and production deficits.*
3. *There is one grammar.*

Reasoning. Suppose properties 1 and 2 hold. Then Broca's area is responsible for a function used in both production and reception, and there must be reception faculties not handled by Broca's area. By property 1, those non-Broca reception faculties must correspond to non-Broca production faculties. By property 2, Broca's faculties must be necessary for the comprehension and production of forms the non-Broca faculties cannot handle; symmetrically, the non-Broca faculties must handle forms Broca's faculties cannot. These two systems accept and generate two different sets of strings. If their grammars were identical, the sets of strings they accept would be identical — a contradiction. Therefore property 3 fails.

Suppose properties 2 and 3 hold. By property 2, there are sentence structures Broca's patients can comprehend but not produce. If property 1 held, the features

Broca's patients can comprehend would equal those they can produce — contradicting property 2. Therefore property 1 does not hold.

Suppose properties 1 and 3 hold. Then any function responsible for the reception of a feature is responsible for its production. A person capable of comprehending a feature therefore has the capacity to produce it. But Broca's patients can comprehend many sentence types they cannot produce — a contradiction. Therefore property 2 does not hold. □

19 Predictions

[The following predictions are noted for elaboration in a later version.]

1. A sentence is Update-hard if and only if it is Broca-hard. That is, a Broca's aphasic struggles with comprehension of a sentence that an unimpaired individual does not if and only if the sentence is Update-hard. *[Relationship to conduction aphasia comprehension deficits to be addressed.]*

2. Neuroscientific analysis of Broca's area will reveal that SPLIT is a core functionality, activated during production; and conversely, that MERGE is activated during reception.

3. Every language has at least one parseable non-CFG structure. *[May need to be rephrased depending on formalisation.]*

4. The frequency of non-CFG structures in speech decreases to vanishing as the level of monitoring decreases — fully spontaneous speech does not contain novel or recursively nested non-CFG structures.

5. Relatedly, non-CFG structures that are easy to parse are never acquired.

6. Acquisition order aligns with the order in which closed-trace chunks can be built when merging in right-to-left order.

7. There are no Update-hard features that can be acquired and that require monotonic right-to-left merging extending beyond maximum stack length. *[May be difficult to instantiate precisely as no such structure appears to exist in any language.]*

8. Update-hard, Merge-hard sentences are extremely difficult to comprehend even for unimpaired individuals — for example, sentences containing centre-embedding over a distance exceeding maximum stack length.

9. Contrapositive of the above: all demonstrably grammatical sentences that unimpaired speakers consistently fail to parse are of this form.

10. Update-hard, Merge-hard sentences should be considerably easier to parse in written form than in spoken form, as written text provides an external memory store extending beyond Stack capacity. We predict significantly higher comprehension performance on such sentences when read rather than heard.

11. We cannot rule out the possibility of nested cross-serial dependencies that remain parseable by the Linear System by virtue of maintaining semantic coherence. If such structures could be constructed and shown to be accepted as grammatical by speakers of CSD-compatible languages, this would constitute strong evidence against mildly context-sensitive grammar as a constraint on the Linear System. *[Requires access to a native speaker of Dutch or Swiss German to construct test cases.]*

12. Every canonically left-branching language has a $\triangleright$-frame relationship — or failing that, some other feature that permits right-branching generation in the Tree System.

20 Not Yet Completed

The following sections are planned but not yet written:

- The strongest possible argument for CSG over CFG for the Linear System.
- A deeper account of the relationship between Time-Manner-Place ordering, pragmatic word order, and childhood concept acquisition order.
- A comparison of right-to-left parsing acquisition order predictions with the language acquisition literature.

21 Open Problems

The following problems are likely beyond the scope of the current paper but represent natural extensions of the model:

- Assuming right-hemisphere homologues of Broca's and Wernicke's areas perform right-hemisphere analogues of Split and Map, what would this mean formally? We strongly suspect this indicates that prosody is a tree-like feature

operated by the right hemisphere — and that this provides a clean account of tip-of-tongue states.

- A formal account of speech intrusion beyond the outline given in Section *[ref: Deliberate Speech]*.
- Construction of a Regular Language grammar for at least one language — probably English — that accounts for Linear System grammaticality judgements.
- Empirical studies on the predictions listed above.
- Why do conduction aphasia comprehension failures not match those of Broca's aphasia exactly, but only partially? Can these differences be derived from the model?

References

[1] Alt, H. (1979). Lower bounds on space complexity for context-free recognition. *Acta Informatica*, 12, 33–61. https://doi.org/10.1007/BF00264016

[2] Engelfriet, J. (1975). Bottom-up and top-down tree transformations — a comparison. *Mathematical Systems Theory*, 9, 198–231.

[3] Miller, G. A. (1956). The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2), 81–97.

[4] *[Full citation needed for the Tandfonline study on sentence length in Wernicke's aphasia: https://www.tandfonline.com/doi/abs/10.1080/02687038.2020.1739616]*

[5] *[Full citation needed for the NCBI Bookshelf entry on conduction aphasia and paraphrase preservation: https://www.ncbi.nlm.nih.gov/books/NBK537006/]*

[6] *[Full citation needed for the PubMed study on conduction aphasia comprehension deficits overlapping with Broca's: https://pubmed.ncbi.nlm.nih.gov/7333108/]*

[7] Li, C. N., & Thompson, S. A. (1976). Subject and topic: A new typology of language. In C. N. Li (Ed.), *Subject and Topic* (pp. 457–489). Academic Press.

[8] Yngve, V. H. (1960). A model and an hypothesis for language structure. *Proceedings of the American Philosophical Society*, 104(5), 444–466.

[9] Kemper, S. (1987). Syntactic complexity and elderly adults' prose recall. *Experimental Aging Research*, 13(1), 47–52.

[10] Weist, R. M. (1991). Spatial and temporal location in child language. *First Language*, 11(32), 253–267.

[11] Garrett, M. F. (1975). The analysis of sentence production. *Psychology of Learning and Motivation*, 9, 133–177.

[12] Garrett, M. F. (1980). Levels of processing in sentence production. In B. Butterworth (Ed.), *Language Production Vol. 1: Speech and Talk* (pp. 177–220). Academic Press.

[13] Potter, M. C., & Lombardi, L. (1990). Regeneration in the short-term recall of sentences. *Journal of Memory and Language*, 29(6), 633–654.